

① Quickstart : LangGraph Essentials - TypeScript

~~Module~~ Module 0: Welcome to the course

- LangGraph is a ~~framework~~ low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful agents.
- LangGraph features
 - latency
 - parallelization - to save actual latency
 - streaming - to save perceived latency
 - long-running agents
 - checkpointing - to reduce the cost of each retry
 - non-deterministic
 - Human-in-the-loop (HITL) - to collaborate with the user
 - tracing, observation, and evaluation (LangSmith)
- Basics :
 - states
 - nodes
 - edges
- Prerequisites
 - Node.js 20+
 - npm
 - OpenAI API key
 - Anthropic API key
 - LangSmith API key
- Running examples : `pnpm tsx src/L1/01-simple-node.ts`
`pnpm dev` (email workflow)
- Install : `pnpm install`

Module 1: Course Overview

- Lesson 1: Nodes and state
 - Components and capabilities
 - State : data
 - Node : functions
 - Edges : control flow
 - serial, parallel
 - conditional
 - Checkpointing / memory
 - Human-in-the-loop (HITL) : Interrupts

② • state is applied to the graph and updated by the ~~graph~~ and returned to the user

• graphs are stateless

• Define graph

1) State (zod) → initialized when the graph is invoked

2) Nodes → ~~use~~ and update state
(function; takes state, returns updated state)

• import { StateGraph, START, END } from '@langchain/langgraph'
import z from 'zod'

```
const state = z.object({
  nlist: z.array(z.string()) // custom props
})
```

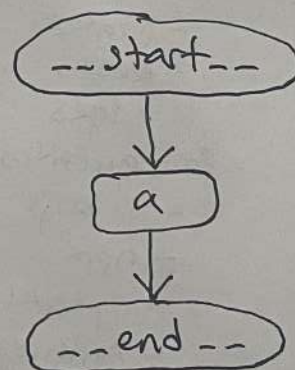
```
type State = z.infer<typeof state>
```

```
function nodeA(state: State) {
  // access state: state.nlist
  return { nlist: [...state.nlist] } // updated state
}
```

```
export const graph = new StateGraph(State)
  .addNode('a', nodeA)
  .addEdge(START, 'a')
  .addEdge('a', END)
  .compile()
```

```
const initialState: State = {
  nlist: ['...']
}
```

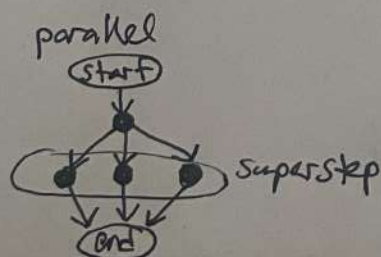
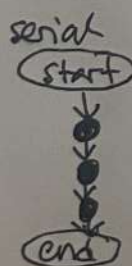
```
const result = await graph.invoke(initialState)
```



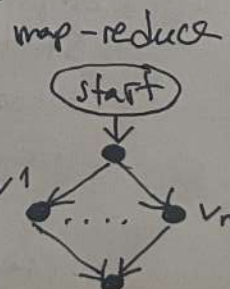
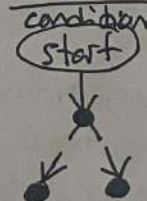
• Lesson 2: Edges

- Edges control the flow in graphs

Edge



Conditional edge



③

• Reducer :

- takes many inputs and merges them
 - values are appended to the list rather than overriding the value
 - reducer can be defined by yourself

```

- z.object({
  nlist: z.array(z.string())
  .register(registry, {
    reducer: {
      reducer: {
        fn: (a, b) => a.concat(b)
      }
    }
  })
})

```

type
reducer function

- import { registry } from '@langchain/langgraph/zod'

```

const state = z.object({
  nlist: z.array(z.string())
  .register(registry, {
    reducer: {
      fn: (left: string[], right: string[]) =>
        left.concat(right)
    }
  },
  default: () => []
})

```

existing state
incoming state

type State = z.infer<typeof state>

• Defining parallel execution of nodes

```

.addEdge('a', 'b')
.addEdge('a', 'c')

```

- Supersteps need to complete before moving on to the next

→ final result: ['A', 'B', 'C', 'BB', 'CC', 'D']

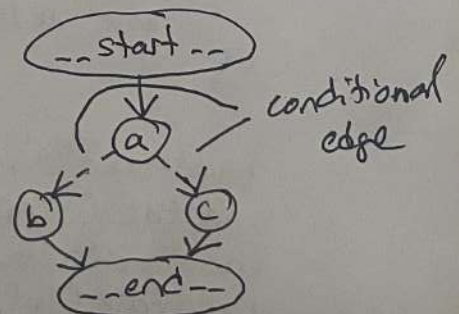
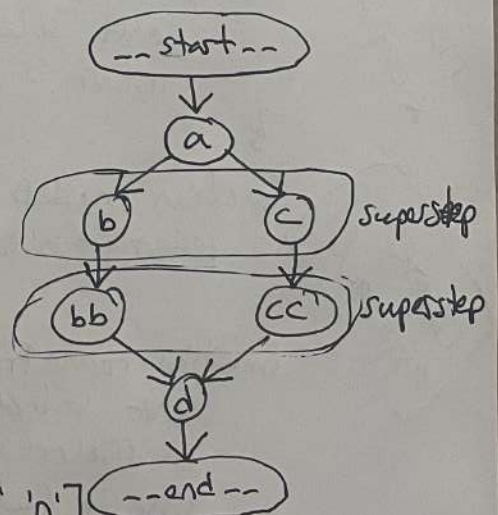
Lesson 3:

- Conditional edge: hands off control to a node based on some existing condition

```

function nodeA(state: State): Command {
  const select = state.nlist.at(-1)
  let nextNode: string
  if (select === 'b') nextNode = 'b'
  else if (select === 'c') nextNode = 'c'
}

```



④

else if (select === 'q') nextNode = END

else nextNode = END

return new Command({

update: { nlist: [select] },

goto: nextNode

← important

})

}

→ conditional edge options to go to

• .addNode('a', nodeA, { ends: ['b', 'c'] })

...

↳ statically define which nodes we're handing off control to, which is useful information for the runtime

• .addEdge(START, 'a')

• .addEdge('b', END)

• .addEdge('c', END)

Note: no edges from 'a' to 'b' and 'a' to 'c'.

• Alternative way to define conditional edges is through the use of the add conditional edges function

function nodeA(state: State): Partial<State> {
 const select = state.nlist.at(-1) ?? ""
 return { nlist: [select] }
}

}

function nodeB(state: State): Partial<State> {
 return { nlist: ['b'] }
}

}

function routeFromA(state: State): string {
 const select = state.nlist.at(-1)
 if (select === 'b') return 'b'
 else if (select === 'c') return 'c'
 else if (select === 'q') return END
 else return END
}

}

...
• .addNode('a', nodeA).addNode('b', nodeB).addNode('c', nodeC)

• .addEdge(START, 'a')

• .addConditionalEdges('a', routeFromA)

• .addEdge('b', END).addEdge('c', END).compile()

more verbose & complex

router approach

⑤ Lesson 4: Memory / checkpoints

- In a superstep, all the active nodes in a step are completed and their values are stored to the state before continuing on to the next node.
- LangGraph allows to specify when the right-hand node executes.
- Nodes execute in sequence, per step
- State is provided to the node at the start of a step and updated at the end of a step.
- State is stored to a checkpoint (memory) at the end of a step
- A thread identifies a collection of checkpoints.
Entire history of the state at each execution step or super step.
- Benefits
 - 1) Recover gracefully from failures - resume without losing progress.
 - 2) Time travel - roll back to a known good point and continue forward.
 - 3) Persistent state - data is preserved even when the graph is not running.
 - 4) Restore state of any step - pick up execution from where you left off.
- import { Command, MemorySaver }
from '@langchain/langgraph'
- const memory = new MemorySaver() → for debugging,
also allows interrupts
- ...
- . compile({ checkpointers: { memory } })
- LangGraph provides a number of checkpointers that work with common persistence layers (Postgres, MongoDB)
- Threads are used to organize memory (UUID)
const config = {
 configurable: { thread_id: threadId }
}
const result = await graph.invoke(inputState, config)

⑥ Lesson 5: Interrupts, Human-in-the-Loop (HITL)

- Interrupt will pause execution and suspend the graph, then continue when we receive the external input.

- import { interrupt } from '@langchain/langgraph'

```
function nodeA(state: State) {  
  const select = state.nlist.at(-1)  
  let nextNode: string  
  if (select === 'b') nextNode = 'b'  
  else if (select === 'c') nextNode = 'c'  
  else if (select === 'q') nextNode = END  
  else {  
    const admin = interrupt({  
      message: `Unexpected input ${select}! Continue?`  
    })  
    if (admin === 'continue') {  
      nextNode = 'b'  
    } else {  
      nextNode = END  
      return new Command({  
        update: { nlist: ['q'] },  
        goto: nextNode  
      })  
    }  
  }  
  return new Command({  
    update: {},  
    goto: nextNode  
  })  
}
```

stop
function
execution

```
function hasInterrupt(result: any): result is { __interrupt__: any } {  
  return result && result.__interrupt__ && Array.isArray(  
    result.__interrupt__  
  )  
}
```

```
...  
let result = await graph.invoke(inputState, config)  
if (hasInterrupt(result)) {  
  const interruptMessage = result.__interrupt__.at(-1)
```


7

```
const msg = interruptMessage.value ? newMessage || 'continue?'
const human = await get userInput(`\n${msg}: `)
```

```
result = await graph.invoke(new Command({ remove: human }
                                config))
```

• when we receive the external input, the entire node function has to run again.

• Lesson 6: Email ^{support} application

1) Define ~~the~~ workflow / graph

- Nodes
- Edges (non-conditional only)
- state
- Memory

2) Implement node functions ^{parallel (superstep)}

- return $\{ \}$ if no state update is needed
- use structured output with models
- wrap errorprone parts with try-catch

→ log error and throw again, return fallback classification/result, use a fallback model or retry

- for optional state props, use default value as fallback when trying to access them in subsequent nodes where the value should always be set

const classification = state.classification ?? defaultClassification

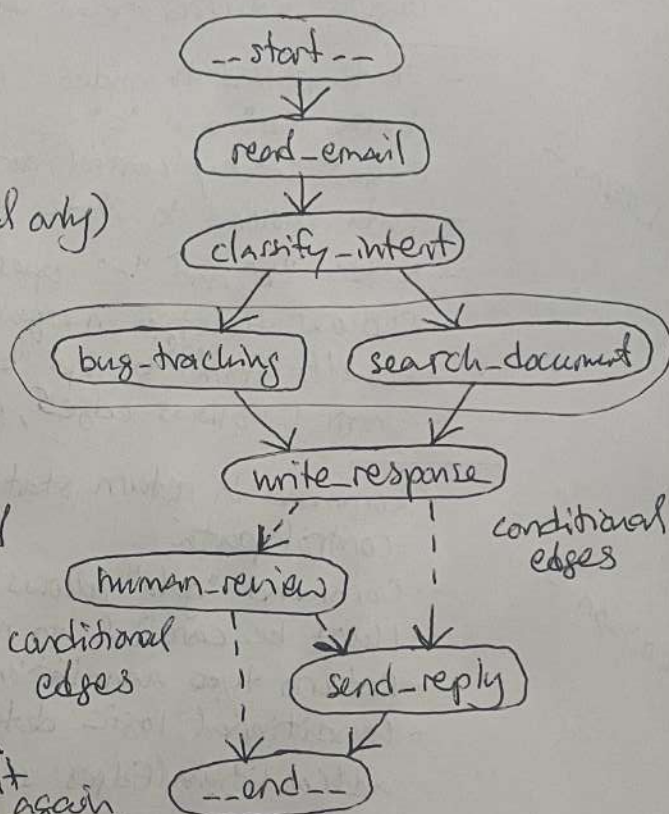
- in case of errors, return an "unknown" value to the state prop that the node was supposed to return
- with conditional edges, return command with dynamic goto in the nodes leading to conditional edges

3) ^{update} ~~the~~ workflow / graph

- Conditional edges ("ends" prop)

(longSmith debugger)

- Run the graph : npm dev → Run LongGraph server



⑧ • Takeaways

Lesson 1

- State : All nodes can share the same state
- Nodes are just functions
- Runtime initializes input state and determines which nodes to run
- Node receives state as input and updates state with return value
- Graph returns final value of state

Lesson 2

- State passed to nodes "bb" and "cc" is the result of both "b" and "c"
- Edges convey control, not data
- Data is stored to state from all active nodes at end of step
- Nodes "b" and "c" operate in parallel
- Reducer function merges values returned
- Results from ^{nodes} "b" and "c" are stored before starting "bb" and "cc"
- Control follows edges, not data

Lesson 3

- Command in return statement updates both state and control path
- Command "goto" allows you to name the next node
- Must be careful to match destination node name
- Return type annotation helps with type checking
- Conditional logic determines the execution path
- addConditionalEdges separates routing logic from node logic
- Router function receives state and returns next node name
- Node function only updates state, not control flow
- Cleaner separation of concerns compared to Command approach
- Router function must return one of the specified destinations

Lesson 4

- Each thread maintains its own separate state

Lesson 5

- interrupt() statement pauses operation and returns value in --interrupt-- field
- When graph is invoked with Command containing resume, operation continues
- Node is restarted from the beginning
- Checkpointer replays responses to interrupts
- Enables human oversight and intervention in automated workflows